



扫码
查看更

 **系列导航：** 上一篇 [07.产品思维] 帮你产出了一份「一页纸 MVP 定义」。这一篇带你把这张纸翻译成一张能照着干的『东西怎么走』的图——架构思维。

兄弟们，今天聊一件反直觉的事——上一篇我们做完了一页纸 MVP，你是不是觉得可以撸袖子开干了？

先别。

我自己做 fuxi 知识库时就吃过这亏。最早我也觉得 06 想清楚意义、07 列好 MVP 就够了，功能也想得明明白白——丢个链接、让 AI 写摘要、存成文件，多简单。

结果做到第三周，存了大几十条素材，真要写文章去翻的时候，傻眼了：我知道这堆文件里肯定有我要的东西，但怎么找？关键词搜？我当时记的词跟现在想搜的词对不上。一条条翻？大几十条还行，几百条就废了。

那一刻我才意识到——我只想清楚了「东西怎么进来」，根本没想「东西怎么出去」。存了一堆，但「取」那一步是个黑洞。

这就是没有架构思维的代价：功能都对，但东西走不通。

所以这一篇，咱们专门补这一课——架构思维。它跟产品思维的关系是这样的：

产品思维回答「我要做什么」，架构思维回答「东西怎么走」。

不写一行代码，但走完之后，你手上会有一张属于你产品架构的「数据流图」——下一篇 09 选具体技术，就是照着这张图选。

看完这一篇，你会拿到：

- 一套普通人 1 小时就能学会的架构思维框架（进、存、取三问）
- 一张 fuxi 知识库的完整数据流图（看它怎么把零散笔记「编译」成能用的知识，含往回织 + 三条「取」的流）
- 一份属于你自己产品的架构文档（动手环节让 AI 帮你出初稿、你审改）

01 架构思维是什么，不是什么

先讲清楚，免得你跟产品思维混了。

架构思维 ≠ 写代码

很多人一听「架构」两个字就犯怵——这不是程序员的活吗？我又不写代码。

说句实话，AI 之前架构确实是工程师的活，因为它跟具体技术绑死——用什么数据库、用什么框架、怎么部署。但 AI 时代，写代码这一层已经被 Claude Code / Codex 接管了，普通人真正稀缺的，反而是「想清楚东西怎么走」的能力。

	产品思维	架构思维	技术思维（下一篇）
回答	我要做什么	东西怎么走	用什么实现
产出	一页纸 MVP	一张数据流图	一份技术选型表

是不是技术活	不是	不是	是
--------	----	----	---

看到了吗？架构思维卡在产品和技术中间，是一座桥——没有它，你的 MVP 永远落不到代码。

类比：盖大楼前的设计图纸

盖一层平房，想到哪砌到哪就行。但要盖 20 层，你敢直接搬砖吗？得先有一张设计图纸——楼分几层、每层干啥，水电怎么通到每一户，人怎么从大堂走到各楼层。这张图纸，就是架构。

这里顺便把产品思维、架构思维、技术思维的区别说清，让你提前心里有个数：

- **产品思维**：决定你要的是平房还是大楼；而 MVP 是——即便你要盖的是大楼，也得先把第一层盖起来住人；
- **架构思维**：这栋楼的总设计师，定下东西在楼里怎么走、能盖到多少层、牢不牢固；
- **技术思维**：哪一层用电钻、什么阶段上吊车——具体怎么施工。

做产品一模一样：

- 做个小工具（一个 Prompt、一个脚本）——不用架构，想到哪做到哪；
- 但你要做的是个有进、有存、有取的「系统」（知识库就是），那就跟盖高楼一样：先画图纸，再搬砖。

而这张图纸，要画清楚两样东西：

- **楼分几层、每层干啥** → 对应「存」（东西存哪、怎么分层）
- **水电、人怎么在楼里走** → 对应「进」和「取」（东西从哪进、怎么走到要用的人手里）

💡 **一句话总结**：架构思维不是技术活，是「动手前先想清楚那张设计图纸」——小工具不用，但凡做个能跑的系统都得先想。而且 AI 时代不用真画图：图纸以前给人看，现在给 AI 看，一份清晰的架构文档，AI 就全懂了。

02 普通人画架构图的最小知识：进、存、取三问

好，现在动手。普通人画架构图，只需要回答三个问题——

进：东西从哪来？ **存**：东西存哪、怎么组织？ **取**：东西怎么出去、给谁用？

就这三个字。听起来简单到你怀疑，但大多数失败的产品架构，都是这三问里至少漏了一问。

进——东西从哪来

具体问三层：

- 1 **来源**：链接、文件、用户手动输入、定时拉取、外部接口推送……
- 2 **触发方式**：用户主动丢进来 / AI 定时去抓 / 别的系统推过来
- 3 **格式**：纯文本、PDF、网页、图片、视频、音频

举个例子——做一个周报工具：

维度	答案
来源	我每天的工作日志

触发	每周五下午定时整理
格式	一段一段的文字

存——东西存哪、怎么组织

这一层别想数据库——那是 09 技术思维要回答的。架构思维只回答「分几层、每层存什么」。

经典分层是三层（你 06 篇见过这套）：

- 原料层（raw）：原始资料，只读不改，事实来源
- 加工层：AI 处理过的结构化成品，可读可写
- 索引层（index）：导航目录，方便快速定位

为什么要分层？因为不分层会出大问题——你 AI 处理一遍写到文件里，再处理一遍又覆盖了，原料就没了，错了就没法回退。分层 = 给数据上保险。

取——东西怎么出去、给谁用

这是普通人最容易漏的一问——我开头讲的 fuxi 早期翻车，就是只想了进和存，没想取。

「取」也有三层：

- 1 谁来取：你自己 / AI 自动触发 / 别人通过分享链接
- 2 什么时候取：随手问 / 定时推送 / 触发某条件
- 3 取什么形式：自然语言问答 / 一份报告 / 一条通知

铁律：画架构图先画「取」——倒推。你倒着想清楚最终输出什么样，才知道前面要存什么、怎么存。

通用模板长这样

下面这张是「进存取」的通用模板图——后面所有架构图，都是这张图的变种：



怎么读这张图：左边是「进」（输入源），中间是「存」（分层结构），右边是「取」（输出方式）。三块串起来就是一个完整的产品数据流。

💡 一句话总结：进、存、取三问——先倒推「取」，再想「存」，最后定「进」，这是普通人画架构图的最小知识。

03 用 fuxi 走一遍：从一页纸到完整数据流

光看模板不够，咱们拿 07 篇定的 fuxi 一页纸 MVP，实打实走一遍架构思维。

先说清楚——fuxi 只是个例子，你要学的是「进存取三问怎么走」这套动作。你自己的产品分几层、有几条流，都会跟 fuxi 不一样，照着方法走你自己的就行。

先把 07 的一页纸 MVP 翻回来

格子	内容
用户	我自己
问题	素材散落、选题时找不到、想发散没人帮、AI 完全没参与
输入	我手动丢进去的链接、想法
处理	Claude 读完 → 写带立场的摘要 → 更新索引 → 反向更新相关条目
输出	一个能问、能查、能告诉我「今天可以写哪篇」的本地知识库
不做什么	不做数据库、不做配图、不做协作、不做多端、不做音视频识别

下面把这张表，用「进存取」三问展开——

进：fuxi 收什么、怎么进来

- 来源：链接（GitHub / X / 公众号）、PDF、想法
- 触发：我手动丢
- 格式：URL / PDF 文件 / 自然语言

注意一个细节——「想法」也是输入。我每天冒的产品脑洞、写作灵感，不是别人给的、是自己脑子里冒的，也得算知识库的输入源——否则它就是个外部资料库，不是「第二大脑」。

存：fuxi 怎么分层组织

这一层是 fuxi 跟普通收藏夹真正拉开差距的地方。06 篇讲过卡帕西那套骨架，fuxi 在它基础上把「存」分成了四层（你的产品不一定四层，看需要）——目录名也不用记，记住每层装什么就行：

- 原料层：抓回来的原文、PDF，原样存着、只读不改——这是「事实来源」，AI 哪天判断错了，能翻回来核对
- 笔记层：每来一条素材，AI 写一篇带自己判断的读书笔记；我自己冒的想法也单独存一篇（这种没有外部链接，写下来的字本身就是知识）
- 编译层：fuxi 的灵魂就在这一层。它不是把笔记一篇篇堆着，而是把好几篇聊同一件事的笔记，合并成一页「结论」；同一个产品、同一个人，所有提到的地方也都汇总成一页。（这两种「结论页」fuxi 里分别叫「主题页」「实体页」，图上会出现，记不住没关系）
- 选题层：把上面那页「结论」，直接变成「这周能写哪篇」的选题清单

类比：编译 vs 收藏。普通收藏夹是把文章一篇篇塞进文件夹，要用时自己一篇篇翻；fuxi 像请了个研究助理，帮你把聊同一个主题的文章全读完，综合成一页「这件事我的结论」——你之后只读这一页，不用回头翻原文。这就是 06 篇说的那四个字：编译 > 检索。

说个真实的踩坑：fuxi 第一版我压根没有编译层，就是每来一条素材写一篇带判断的笔记（也就是上面的笔记层），以为这就够了。结果写一篇「AI 视频工具怎么选」的稿子时，我发现自己还是得把五六篇相关笔记一篇篇打开、对着看、在脑子里现场拼——笔记是攒下来了，但「我的判断」还散在五六个文件里，每次写都要重新拼一遍。这才逼我加了编译层：让



AI 提前把这五六篇合成一页「我对 AI 视频工具的当前判断」，下次写直接读这一页。所以编译层是 fuxi 第二版才加的，不是一开始就想到的——这正是 04 节要讲的「架构分版本」。

最后再配几个目录文件，帮你和 AI 快速找到东西、记得住每次都存了啥——「存」这一层就齐了。

取：fuxi 怎么输出（三条流）

这是 fuxi 跟卡帕西版本最大的差异——卡帕西的终点是「答得出」，fuxi 的终点是「写得出」。所以 fuxi 顺着 02 讲的「取」三问（谁取、何时取、取什么形式），落出了三条独立的流——你的产品几条不一定，看你「取」的场景有几种：

- 1 问答流：我问什么 → AI 先查目录、找到相关的那页「结论」 → 直接读那页（而不是回头一篇篇翻原文） → 综合回答给我
- 2 选题流：基于主题页，反过来告诉我「这个主题现在能写哪几篇」，直接给一份可执行的选题
- 3 周报流：每周梳理「本周入库了什么」+「哪个主题该编译了」

第 2 条「选题流」是 fuxi 比卡帕西多出来的关键一层——知识库不只回答我的问题，还反过来给我派活。

完整数据流图

把上面三问汇成一张图：



怎么读这张图：左边两路「进」——链接 / PDF 这类外部资料走「单源笔记」，我自己的想法走「想法笔记」；两路汇进中间的编译层，被编译成主题页 / 实体页；再从编译层分出右边三条「取」的流。重点看编译层那条往回织的回环箭头——这是 fuxi 跟普通收藏夹最本质的区别：每加一条新资料，AI 会回头把相关的主题页、实体页都重写一遍，还会标出新旧资料的矛盾，不是简单加个链接。这就是 06 篇说的「往回织」落到架构图上的样子。

跟 07 一页纸的对照

07 一页纸	08 架构图位置
输入：链接、想法	左侧「进」
处理：摘要 + 反向更新	中间「存」的编译层 + 往回织回环
输出：能问、能告诉我写啥	右侧三条「取」的流
不做：协作、多端、配图	图里没有 ≠ 默认不做

看到了吗？架构图本质就是一页纸 MVP 的「流动版」——把静态的功能列表，翻译成动态的「东西怎么走」。

💡 **一句话总结：***fluxi 把笔记「编译」成主题页 + 往回织回环 + 三条「取」的流（问答 / 选题 / 周报）——这是它从「答得出」升级到「用得出」的架构秘密。*

04 普通人最容易踩的四个架构坑

光会画图不够，得知道画错的样子长什么样。下面这 4 个坑我自己全踩过，按出现频率从高到低排——

坑一：先想功能，后想流

症状：列了一长串功能（自动分类、智能推荐、可视化、协作……），然后挨个想怎么做。

后果：功能堆成一堆，**流向混乱**——这个功能输出什么没人接，那个功能要输入没人提供。

治法：先画流，再想功能。功能是流的副产品——流通了，功能自然就稳了；流没通，功能再多也是孤岛。

我做 fluxi 之前，列功能列了 30 多条。但真的画完进存取之后，发现 80% 的功能要么是「流上的某个节点」，要么是「不在流里的孤儿」——后者直接砍掉，剩下的就是核心架构。

坑二：只想「进」不想「取」（最常见，最致命）

症状：想清楚了输入源、想清楚了存哪——然后就开干了。

后果：你做出来的是一个精美的收藏夹，不是知识库。

这是我开头讲的 fluxi 翻车——更扎心的是，连 Notion、Obsidian 这些被无数人推荐的工具，都没逃出这个坑：只想清楚了进和存，从没把「取」想透。这也是 06 篇说它们为什么不够用的根本原因。

治法：画架构图时，**强制要求自己最先画「取」**。你写不出至少 2 条「取」的方式，就不该继续往下做——因为做出来你也用不上。

坑三：抽象层级混乱

症状：架构图上同时出现「微信公众号文章」「Claude API」「JSON 文件」——把数据、工具、技术全混在一起。

后果：你看不清「东西怎么走」，因为图上一半是「东西」、一半是「工具」。

治法：架构图只画「东西」，不画「工具」。「用什么工具实现」是 09 技术思维的事。08 阶段，图上每个框框要么是数据（链接、摘要、索引），要么是动作（抓取、整理、查询）。

坑四：一上来就想完美架构

症状：想把 v1 架构图画到「未来三年都不用改」的程度。

后果：永远画不完。没开始做，先在画图阶段卡死。

治法：架构跟 MVP 一样，分版本。v1 架构图就服务 v1 MVP——能跑通进存取三块就行。v2、v3 的架构是后续的事。

我自己 fluxi 现在的架构图，已经是 v3 了——前两版都是用了一段时间发现哪里不顺，回头改的。架构图是活的，不是一次性的。

💡 **一句话总结：***先画流再想功能、先画「取」再画「进」、只画东西不画工具、架构分版本——四条铁律避开 90% 的架构翻车。*

05 进阶：产品做 MVP，架构想得更全

到这儿你可能犯嘀咕：坑四刚说「别追求完美架构、分版本」，怎么又要「想全、留余地」？

不矛盾——功能和架构，是两套标准：

功能狠狠做减法，架构得尽量想得更全。功能只留最小可用；但架构要算两条——能长大（可扩展）、不会塌（稳定）。

还是那栋大楼：先盖 10 层，但能往上加

前面 01 节盖大楼那张图纸，到这儿正好接上。

你不用一开始就盖到 20 层——先盖 10 层就行（这就是 MVP）。但只要承重骨架立得稳、分层留了往上接的余地，将来要加到 15 层、20 层，在原结构上往上长就行，不用把楼炸了重盖。

而且产品比真盖楼还省事：真盖楼加层受地基限制，但软件架构只要分层清晰、留了接口，加「楼层」（新功能）就是往上接，连地基都不用重浇。

这就是架构思维的扩展性——不要求你一开始想到结局，但要求你立的骨架能长大。具体到画图，就是这三样要往远了想：

- 分层结构：层次先分好，哪怕某一层暂时是空的
- 数据怎么组织：格式、字段留点余量，别写死
- 扩展口：未来可能接入的输入源、输出方式，先在图上留个虚位

fuxi 就是这么干的

还记得那个踩坑故事吗？fuxi 第一版没有编译层——但我的分层结构一开始就给它留好了位置，那层先空着。等 v2 真有加，是直接「往上长」，原料层、笔记层一动没动。

反过来想：假如当初我没分层、把摘要和原始资料全混在一个文件夹里，那加编译层就得把几百条数据推翻重整——那才是灾难。

再比如，fuxi 第一版我只支持 GitHub 链接，但设计时就给「多渠道」留了口子。

你可能会问：不都是链接吗，GitHub 链接和 X 链接能有啥区别？区别大了——不同来源的抓取难度、反爬对抗完全不一样，这是下一篇「技术思维」要聊的事。但架构这一层，先把「多渠道」这个口子留好就行，具体怎么抓是后面的事。

所以「架构留余地」的妙处是：它几乎不增加 v1 的工作量，却能让你 v2 不推翻重来。

楼更得建得牢：数据别丢、错了能回退

光能加层还不够——楼更得建得牢。承重墙不能为了省钱砍掉，不然盖到 15 层塌了，能加层的本事再大也白搭。

落到产品上，稳定性就两条（02、03 节其实埋过）：

- 数据不能丢：原料层只读、永远留着事实来源——AI 哪天判断错了，你能翻回原文核对（这就是 02 说的「分层 = 给数据上保险」）
- 出错能回退：每一步处理别覆盖上一步，留得住「上一个版本」，错了能退回去

可扩展性管「能不能长大」，稳定性管「会不会塌」——架构这两条都想到了，才算考虑全面。

功能克制，架构算两条

怎么对待	
功能（做什么）	狠狠做减法——v1 只留最小可用那几个
架构（怎么走）	想两条：能长大（分层、留扩展口）+ 不会塌（数据只读保险、错了能回退）

注意是适度：不是让你把三年后的功能全画进图里（那又掉回坑四了），而是心里有未来的生长方向、在结构上留好位置，但 v1 不强求实现它。



💡 **一句话总结：**功能做 MVP，架构算两条——能长大（可扩展：加功能往上接，不推翻）+ 不会塌（稳定：数据不丢、错了能回退）。

06 动手：让 AI 帮你出第一版架构

老规矩，动手。但这一篇的动手，跟你想的不一样——

AI 时代，你根本不用真去画架构图。架构图以前是画给工程师看的；现在你的「工程师」是 Claude / Codex，它们读文字比读图快得多。所以你要产出的不是一张图，是一份 AI 一看就懂的「架构文档」，然后让 AI 照着它往下帮你做。

更省事的是：这份文档的初稿，直接让 AI 起就行，你的活儿是审和改。

第一步：把料喂给 AI，让它出初稿

把你 07 篇那张一页纸 MVP，加上这篇学的「进存取三问」，一起丢给 AI。这个 Prompt 直接抄：

纯文本

这是我的产品 MVP 定义：

[贴上你 07 篇的一页纸 MVP]

请按「进、存、取」三个维度，帮我设计一版架构方案：

- 进：东西从哪来、怎么触发、什么格式
- 存：分几层、每层放什么（先别管具体技术）
- 取：谁来取、什么时候、取什么形式（至少给 2 条）

原则：先想流再想功能，只描述「东西怎么走」，不要写用什么工具。

AI 会还你一份进存取齐全的架构初稿。

第二步：对着三问审，哪不对让 AI 改

别全盘照收——AI 给的是初稿，拍板的是你。对着这三问审：

「取」给够 2 条了吗？最容易漏（坑二），AI 也常只给一条

「存」分层合理吗？有没有把原料和成品混在一层（坑三）

每条「进」都能追到某条「取」吗？对不上的，让 AI 砍掉

哪不满意就直接说：「『取』只给了问答，再加一条主动推送给我的」「存这块原料和加工混了，拆成两层」。一步步改，改到你看着这份文档、能在脑子里想象出产品怎么跑起来为止。

第三步：让 AI 整理成一份「架构文档」存好

满意了，让 AI 把最终方案整理成一份干净的 markdown 文档，存进你的项目——这就是你产品设计初期的架构文档。它管两头：

- 给 AI 看：下一篇 09 选技术、之后让 AI 写代码，都照着它来，AI 一看就懂；
- 给自己看：产品长大时回来改它就行——架构是活的（坑四说过）。照样不用自己动手，把想改的地方说给 AI，让它去改。

💡 **一句话总结：**动手不是自己画图，是让 AI 出架构初稿 → 你对着进存取审 → 一步步改 → 存成给 AI 看的架构文档。

07 小结

这一篇核心就一句话：

产品想清楚做什么之后，下一步不是开干，是先理清「东西怎么走」——给 AI 一份看得懂的架构文档。

架构思维三问：进、存、取——

问	解决什么
进	东西从哪来、怎么触发、什么格式
存	分几层、每层放什么（别管具体技术）
取	谁来取、什么时候、什么形式

四个坑别踩：

- 先画流，再想功能
- 先画「取」，再画「进」
- 只画东西，不画工具
- 架构分版本，别追求一次完美

再加一条进阶心法：功能做 MVP，架构算两条——能长大（可扩展）+ 不会塌（稳定）。

到这里，你手上应该已经有了：

- 06：为什么做（动机）
- 07：做什么（一页纸 MVP）
- 08：怎么走（架构 v1）← 你在这

下一篇 09「技术思维：让 AI 帮你选最适合的技术栈」——架构理清了，下一步是用什么工具实现。我会带你看一个反常识的真相：AI 时代选技术，比想象中简单得多——你不需要懂技术，你需要懂的是「怎么让 AI 帮你选」。

留个问题给你：你让 AI 出的那版架构里，「取」写下来几条？不到 2 条，多半是产品定义还没想透——回 07 补补课。

本系列持续更新，建议收藏。